

智能 AED 救援车系统

摘要

针对公共场所心脏骤停患者黄金救援时间短、传统人工取送自动体外除颤器（AED）效率较低的问题，我们团队设计了基于 ROS2 的智能 AED 救援车系统。该系统由主导航、目标检测、跌倒识别、语音交互、远程控制及自动返航等功能于一体，能够在接收到救援任务后快速规划最优路径，自主避障并将 AED 运输至目标位置，为现场急救争取宝贵时间。

本系统以 ELF2 开发板为核心，结合二维激光雷达，实现环境建模、定位导航及障碍物检测；利用深度学习目标检测算法识别人体状态，对疑似跌倒人员进行智能判断；在导航过程中，通过语音播报提醒周围人员避让，并引导现场人员及时取用 AED 实施救援。到达目标点后，小车自动停止报警并等待救援完成，在规定时间内可执行返航任务，提高设备利用率和连续工作能力。

实验结果表明，该 AED 救援小车能够在复杂室内环境中稳定完成自主导航、动态避障、目标识别和 AED 配送任务，具有较高的导航精度、运行稳定性和实时响应能力。该系统能够有效缩短 AED 送达时间，降低人工配送成本，为智慧医疗、智慧校园、医院、商场及大型公共场所的应急救援提供了一种智能化、自动化的解决方案，具有良好的应用价值和推广前景。

关键词：AED；智能救援小车；ROS2；自主导航；跌倒检测；激光雷达；目标识别

第一部分作品概述

2.1 功能与特性

智能 AED 急救小车旨在解决公共场所突发心搏骤停时“黄金四分钟”内的 AED 快速投递问题，化被动的“人找设备”为主动的“设备找人”。其核心功能包括：

跌倒精确识别：通过 rknn 模型接入远程摄像头的视频，实现对患者跌倒的快速识别，同时为了准确性，调高置信度，加入连续十帧触发机制，避免误识别。

自主导航与动态避障：结合 Nav2 导航和 AMCL 定位，小车能在复杂、人流密集的室内外环境中自主规划最优路径，并实现对行人及障碍物的实时动态避让。

实时语音交互：设备内置扬声器，可实现语音播报和警报功能，汇报当前执行情况，并在行程中鸣笛提醒周围行人注意避让。

本小车具备响应速度快、部署灵活、人机交互界面友好等特性，极大提升了公共空间的急救效率。

2.2 应用领域

智能 AED 救援小车主要应用于面积广阔、人流密集且突发心血管疾病风险较高的公共场所。具体包括：

大型交通枢纽：如地铁站（例如徐州地铁网内各站）、高铁站、机场航站楼，解决站内面积庞大、人工寻找固定 AED 耗时过长的的问题。

高校校园与体育场馆：针对学生进行高强度体测（如 3km 长跑或 400m 冲刺）、大型体育赛事及日常操场锻炼时突发心脏骤停事件，提供快速的赛道旁医疗响应。

大型商超与展览中心：在内部结构复杂的商业环境中，小车能够穿梭于人群和货架之间，精准将设备送达求救者手中。

智慧社区与工业园区：作为智慧安防与医疗基础设施的一部分，与园区的监控系统联动，为老年人和工作者提供全天候的生命保障。

2.3 主要技术特点

本系统采用 ROS 机器人操作系统作为软件框架，实现模块化设计与多节点协同运行。

- (1) 基于激光雷达 SLAM 技术完成环境建图与 AMCL 定位，实现高精度自主导航；
- (2) 采用全局路径规划与局部避障算法，能够在复杂环境下安全运行；
- (3) 结合深度学习目标检测技术，实现跌倒人员识别与救援目标确认；
- (4) 支持远程监控与任务调度，可实时查看小车运行状态；
- (5) 具备自动返航与待机功能，提高设备利用率和系统可靠性；
- (6) 采用模块化硬件架构，便于后续功能扩展和维护升级。

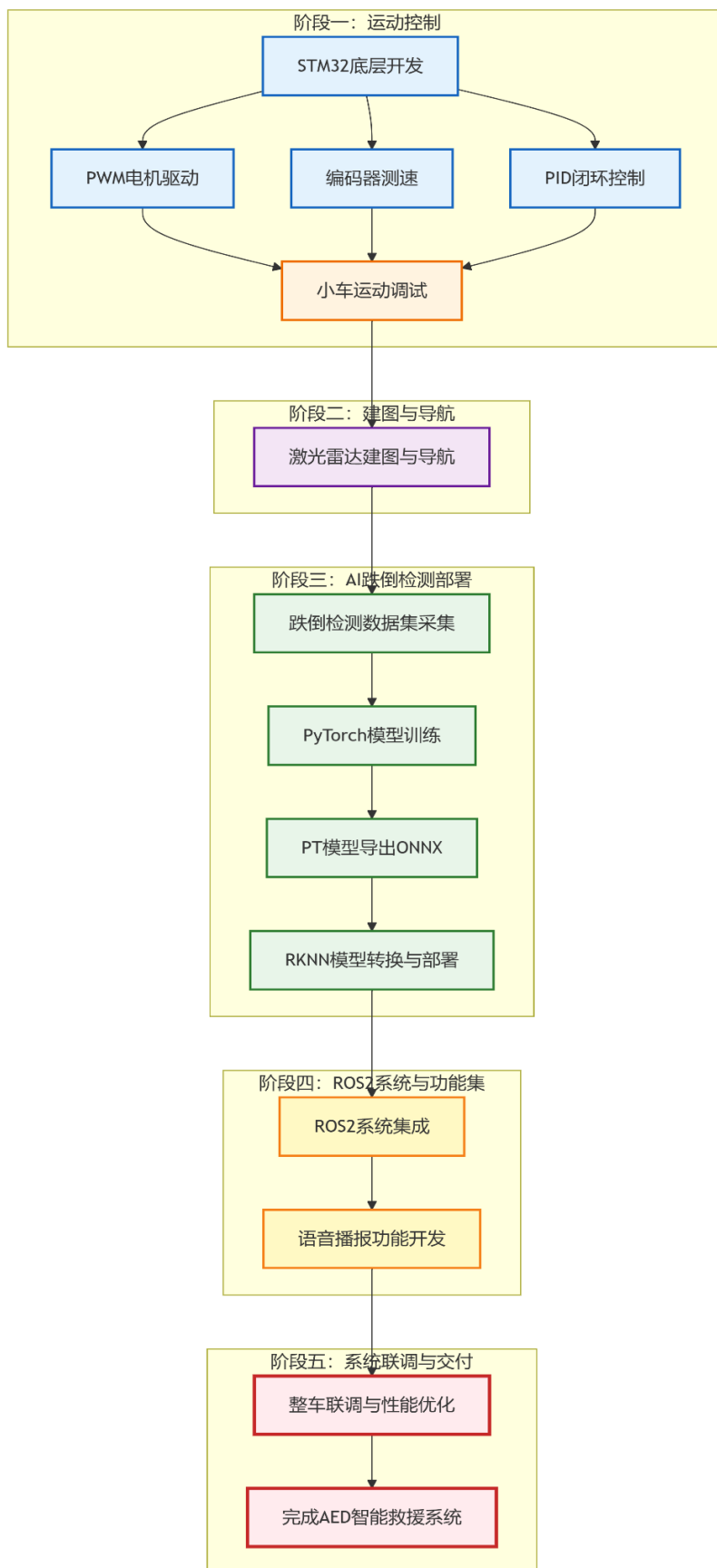
2.4 主要性能指标

指标项目	性能参数
导航方式	激光雷达自主导航
定位精度	$\leq 10\text{cm}$
最大运行速度	$0.8\sim 1.2\text{m/s}$
避障距离	$\geq 0.3\text{m}$
跌倒检测准确率	$\geq 90\%$
自动返航成功率	$\geq 95\%$

2.5 主要创新点

- (1) 将 AED 配送与自主移动机器人技术相结合，实现急救设备智能化配送。
- (2) 融合激光雷达导航与视觉跌倒检测技术，实现“发现—运输—送达”一体化救援流程。
- (3) 设计智能语音引导机制，提高现场群众参与急救的效率。
- (4) 具备自动返航与循环工作能力，提升公共场所 AED 设备利用率。
- (5) 采用 ROS 模块化架构，便于后续功能扩展和智慧医疗系统接入。

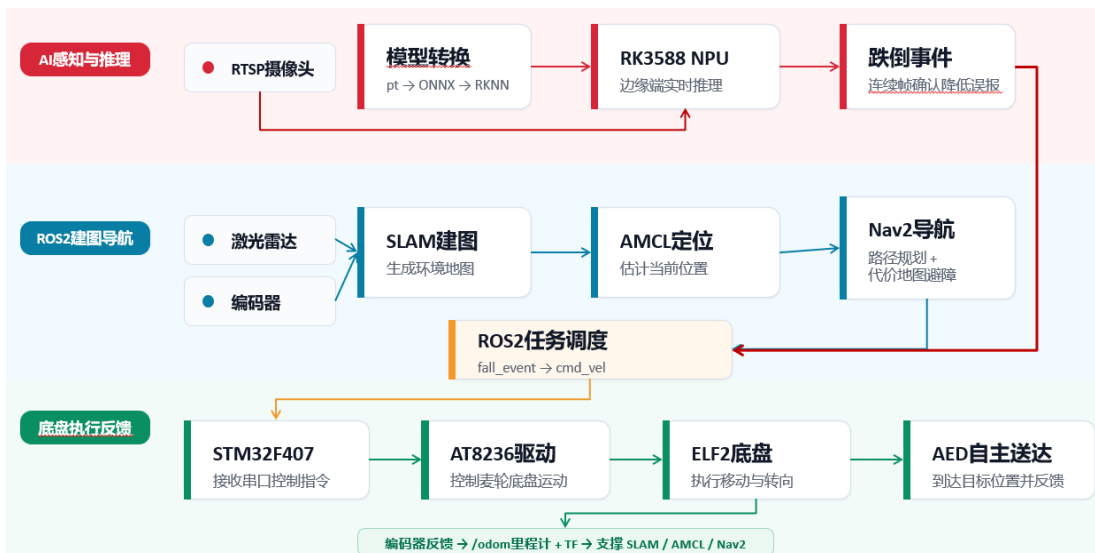
2.6 设计流程



第二部分系统组成及功能说明

2.1 整体介绍

硬件与控制流程：从识别到送达的闭环



2.2 硬件系统介绍

2.2.1 硬件整体介绍

本系统采用"上位机智能决策+下位机运动控制"的分层硬件架构，由ELF2边缘计算平台、STM32运动控制器、激光雷达、驱动模块、动力系统及通信模块组成。

ELF2作为系统上位机，负责运行Ubuntu操作系统及ROS2框架，完成地图构建、自主导航、路径规划、跌倒检测模型推理、任务调度及语音播报等高层功能。视觉算法采用RKNN部署于ELF2内置NPU，实现跌倒检测模型的边缘端实时推理。

STM32作为底层运动控制核心，主要负责底盘运动控制及外设管理，包括PWM输出、电机速度闭环控制、编码器数据采集、超声波测距及各类IO控制。STM32根据ELF2发送的速度指令完成左右轮电机控制，并实时反馈编码器信息，为自主导航提供稳定可靠的运动基础。

电机驱动部分采用AT8236双路直流电机驱动模块，可提供较大的输出电流，支持PWM调速及电机正反转控制。STM32输出PWM及方向控制信号至AT8236，驱动两台520减速直流电机完成差速运动，实现前进、后退、

转向及原地旋转等动作。

环境感知部分采用 LD06 二维激光雷达，实现周围环境扫描与障碍物检测。LD06 通过串口与 ELF2 通信，配合 ROS2 完成 SLAM 建图、AMCL 定位及 Nav2 自主导航，为车辆提供可靠的环境感知能力。

ELF2 与 STM32 之间采用 TTL 转 USB 通信模块进行串口数据交互。STM32 通过 USART 发送底盘状态、编码器速度等信息，ELF2 接收导航算法计算得到的速度控制指令，并将其发送至 STM32 执行，实现上下位机之间稳定、高效的数据通信。

系统动力部分采用四台 520 减速直流电机作为驱动机构，配合编码器实现闭环速度控制，提高车辆运动精度及导航稳定性。整车采用大容量锂电池集中供电，并通过降压模块为激光雷达提供稳定电源，保证系统长时间稳定运行。

2.2.2 机械设计介绍（如果有的话，从总体到局部，逐级给出各组件的具体设计图，**可以是 CAD 文件截图或者手绘图片**）；

车壳采用 3D 打印的方式，分开打印车壳和 AED 底座、AED 模型，再将三者组装到一起。其中，车壳上方预留了雷达的孔位，车后侧预留了和 AED 底座的螺丝连接口，侧面增加了散热口。AED 模型通过磁吸方式与 AED 底座连接。摄像头外壳也采用了 3D 打印，起到一个保护且更加美观的效果。

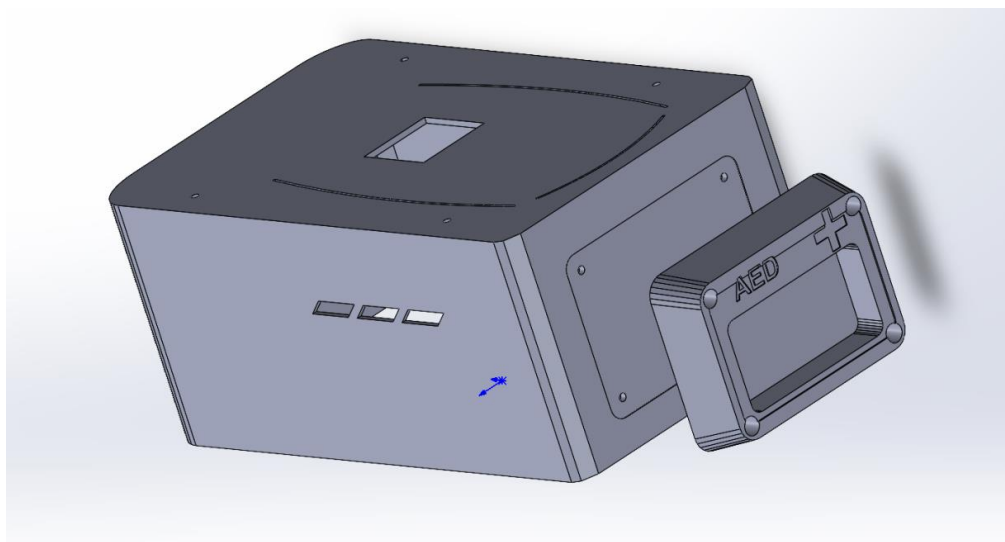


图 2.2.2.1 车体设计

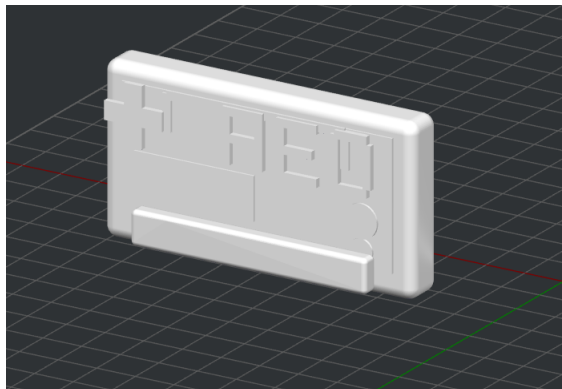


图 2.2.2.2.AED 模型设计

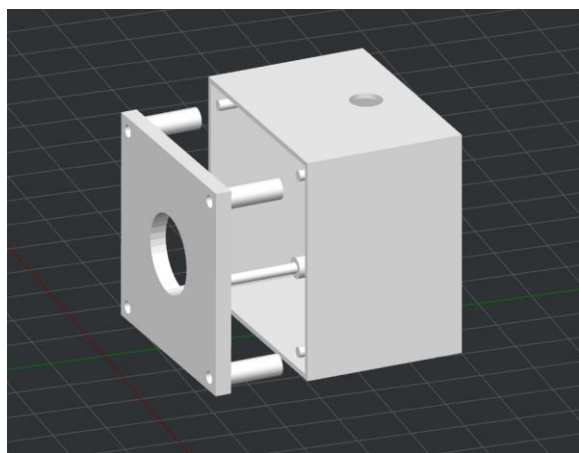


图 2.2.2.3 摄像头外壳设计

2.2.3 电路各模块介绍（从总体到局部，逐级给出各模块的具体设计图，并标记出关键的输入、输出信号线，**可以是电路图、SCH 原理图、PCB 版图等截图**）；

1. 总体连接关系

整车电路可以按“上位机计算、底层控制、感知供电、动力执行”四层理解。ELF2 负责接收 LD06 雷达数据并运行导航/上位机逻辑；STM32F407ZGT6 通过 TTL 转 USB 接收 ELF2 指令；AT8236 根据 STM32F407ZGT6 的 A/B 路控制信号驱动 520 马达，同时将执行状态、故障或采样信息回传给 STM32F407ZGT6。

总体电路连接设计图



图 2.2.3.1 总体电路连接设计图：电源、数据、控制、反馈与电机输出主线

模块	关键输入	关键输出	主要作用
ELF2	LD06 雷达数据； STM32F407ZGT6 回传状态	经 TTL 转 USB 下发运动/控制指令	作为上位机和导航计算平台，完成感知数据接收、路径/任务逻辑和底盘指令下发。
TTL 转 USB	ELF2USB； STM32F407ZGT6UART TX/RX/GND	USB 与 TTL 串口数据互转	把 ELF2 的 USB 通信转换为 STM32F407ZGT6 可用的 UART 串口通信。
STM32F407 ZGT6	TTL 串口指令；四路编码器反馈	TIM3/TIM4 共 8 路 PWM；USART1 状态回传	作为底层控制器，解析上位机指令，执行 20msPID 闭环并控制四个电机。
AT8236	STM32F407ZGT6PWM 控制信号；电机电	四个 520 马达功率输出	完成电机功率驱动，将低功率 PWM 控制信号转

	源 VM/GND		换为马达驱动电流。
LD06	稳压模块 VCC/GND	雷达扫描数据线至 ELF2	提供激光雷达扫描数据，供 ELF2 进行定位、建图或导航。
稳压模块/ 电池	电池输入	稳定电源给 LD06 等负载	提供稳定供电，降低电池电压波动对雷达工作的影响。

(1). ELF2 与 STM32F407ZGT6 串口通信

ELF2 与 STM32F407ZGT6 之间使用 TTL 转 USB 模块建立串口链路。STM32F407ZGT6 属于 STM32F407 系列，具备多路 USART/UART、定时器 PWM 和丰富 GPIO，适合承担底盘控制、驱动输出和状态采集任务。

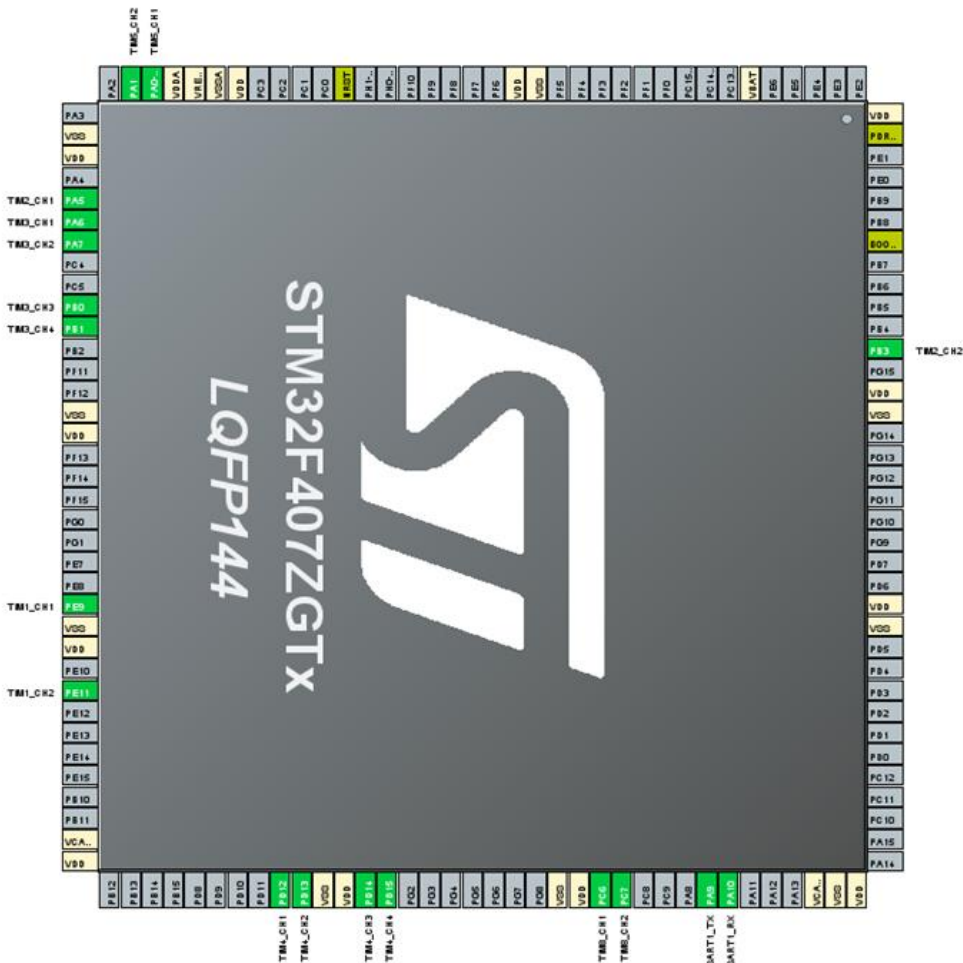


图 2.2.3. (1). 1STM32F407ZGT6 使用引脚

电机	STM32F407ZGT6 引脚/复用	连接对象	具体功能
左前电机	PD12/TIM4_CH1; PD13/TIM4_CH2	AT8236 模块 1AIN1/AIN2	正转 PWM 输出 反转 PWM 输出
左后电机	PD14/TIM4_CH3; PD15/TIM4_CH4	AT8236 模块 1BIN1/BIN2	正转 PWM 输出 反转 PWM 输出
右前电机	PA6/TIM3_CH1; PA7/TIM3_CH2	AT8236 模块 2AIN1/AIN2	反转 PWM 输出 正转 PWM 输出
右后电机	PB0/TIM3_CH3; PB1/TIM3_CH4	AT8236 模块 2BIN1/BIN2	反转 PWM 输出 正转 PWM 输出

表（1）.1 PWM 输出引脚分配

电机	STM32F407ZGT6 引脚/复用	连接对象	功能
左前轮（LF）	TIM1	CH1	PE9
左前轮（LF）	TIM1	CH2	PE11
右前轮（RF）	TIM2	CH1	PA5
右前轮（RF）	TIM2	CH2	PB3
左后轮（LR）	TIM5	CH1	PA0
左后轮（LR）	TIM5	CH2	PA1
右后轮（RR）	TIM8	CH1	PC6
右后轮（RR）	TIM8	CH2	PC7

表（1）.2 编码器接口分配

外设	GPIO	功能
USART1	PA9	TX（串口发送）
USART1	PA10	RX（串口接收）

表（1）.3 USART1 串口接口分配

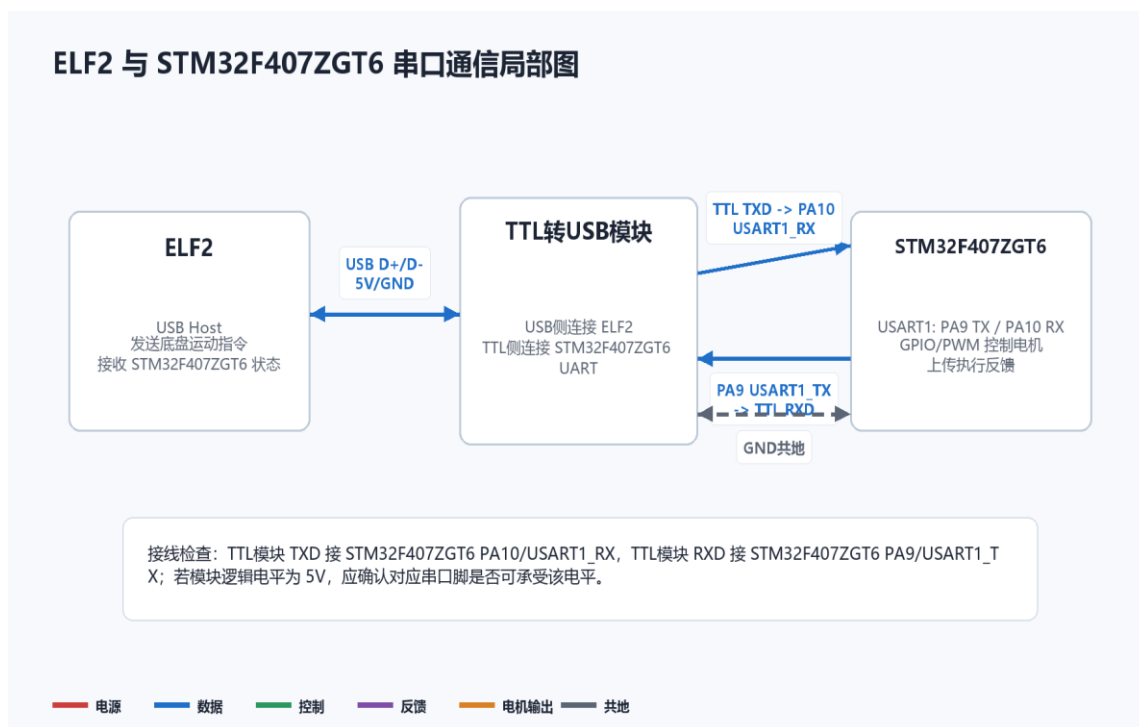


图 2.2.3.1(1).2 ELF2、TTL 转 USB、STM32F407ZGT6 通信局部设计图

信号线	方向	连接关系	设计要点
USB	ELF2→TTL 转 USB	ELF2USB 口连接 TTL 转 USB 模块 USB 侧	用于把上位机通信接入串口转换模块。
TXD/RXD	双向	TTL （ TXD ） 连 接 STM32F407ZGT6 (RXD)PA10 ； STM32F407ZGT6 (TXD)连接 TTL (RXD)	串口收发需要交叉连接，并保证波特率、数据位、校验位一致。
GND	公共参考	TTL 转 USBGND 与 STM32F407ZGT6GND 相连	串口通信必须共地，否则电平参考不一致会导致通信异常。

表（1）.4 TTL-USB 连接关系

主文件中 USART1 配置为 115200baud、8 数据位、1 停止位、无校验。
STM32F407ZGT6 每 20ms 执行一次闭环控制，并以 20ms 周期向 ELF2 上传四路编码器增量。

通信方向	帧格式	载荷含义	用途
ELF2→STM32F407ZGT6	AA55+vx (2B)+vy (2B)+w (2B)+S UM+OD	目标前后速度、横向速度、角速度	接收 ROS2/上位机下发的底盘运动指令，并刷新 500ms 看门狗。
STM32F407ZGT6→ELF2	AA66+LF/RF/LR/RR 编码器增量各 2B+SUM+OD	四个 520 马达的编码器原始增量	用于上位机获得底盘执行反馈，支持里程计、速度闭环或状态监控。

表（1）.5 ELF2-STM 交互信息

（2）. STM32F407ZGT6、AT8236 与 520 马达模块

STM32F407ZGT6 作为底盘控制核心，向 AT8236 输出电机控制信号；AT8236 负责功率放大并驱动 520 马达。主文件显示底盘为四轮闭环结构：TIM4 的 4 路 PWM 控制左前/左后，TIM3 的 4 路 PWM 控制右前/右后；每个 520 马达占用一对 AT8236 输入脚。

AT8236 芯片资料显示其为单通道直流有刷 H 桥驱动，核心控制脚为 IN1/IN2，功率输出为 OUT1/OUT2；若使用双路电机驱动模块，则四轮底盘通常需要两块双路模块或等效四路驱动。主文件中的闭环反馈主要来自四路编码器：TIM1、TIM2、TIM5、TIM8 分别读取 LF、RF、LR、RR 的编码器增量，并通过 USART1 回传给 ELF2。

STM32F407ZGT6、AT8236 与四路 520 马达局部图



图 2. 2. 3. 1 (2). 1 STM32F407ZGT6、AT8236 电机驱动与 520 马达局部设计图

定时器	功能
TIM1	左前轮编码器
TIM2	右前轮编码器
TIM3	右侧电机 PWM 输出
TIM4	左侧电机 PWM 输出
TIM5	左后轮编码器
TIM8	右后轮编码器

表 (2). 1 STM32 定时器资源分配

外设	GPIO	功能
TIM5_CH1	PA0	左后轮编码器 A 相
TIM5_CH2	PA1	左后轮编码器 B 相
TIM2_CH1	PA5	右前轮编码器 A 相

TIM3_CH1	PA6	右前轮 PWM 反转
TIM3_CH2	PA7	右前轮 PWM 正转
USART1_TX	PA9	串口发送
USART1_RX	PA10	串口接收
TIM3_CH3	PB0	右后轮 PWM 反转
TIM3_CH4	PB1	右后轮 PWM 正转
TIM2_CH2	PB3	右前轮编码器 B 相
TIM8_CH1	PC6	右后轮编码器 A 相
TIM8_CH2	PC7	右后轮编码器 B 相
TIM4_CH1	PD12	左前轮 PWM 正转
TIM4_CH2	PD13	左前轮 PWM 反转
TIM4_CH3	PD14	左后轮 PWM 正转
TIM4_CH4	PD15	左后轮 PWM 反转
TIM1_CH1	PE9	左前轮编码器 A 相
TIM1_CH2	PE11	左前轮编码器 B 相

表（2）. 2 STM32 GPIO 资源汇总

(3). LD06 激光雷达供电与数据模块

LD06 的供电链路由电池先进入稳压模块，再由稳压模块给雷达提供稳定电源；雷达数据线直接连接 ELF2，用于给上位机提供扫描数据。电源线和数据线在图纸上应分开标注，避免把供电链路和数据链路混在一起。

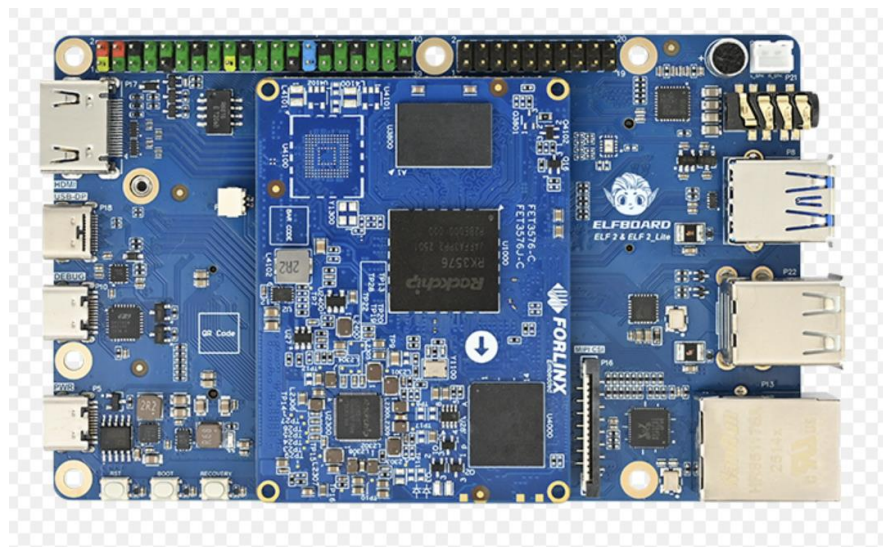
LD06 激光雷达供电与数据局部图



图 2.2.3.1(3).1 激光雷达供电与数据局部设计图

链路	输入端	输出端	关键检查点
雷达供电	电池	稳压模块输入	确认输入电压范围满足稳压模块要求。
稳定输出	稳压模块	LD06VCC/GND	输出电压和电流能力应满足 LD06 工作要求。
雷达数据	LD06 数据线	ELF2 雷达数据接口	标注数据方向，并确认接口协议、波特率或驱动配置一致。
公共地	稳压模块 GND	LD06/ELF2/STM32F407 ZGT6 系统地	建议最终共地，减少数据线参考电平不一致问题。

2. 主要电路原理图



		P26			
VCC 3V3	3.3V	1	2	5V	VCC 5V
I2C4-I2C4-SDA 3V3	SDA.1	3	4	5V	VCC_5V
I2C4-I2C4-SCL 3V3	SCL.1	5	6	GND	GND
GPIO2_C6-GPIO3_B3	GPIO.7	7	8	TXD	UART6-UART9-TX
GND	GND	9	10	RXD	UART6-UART9-RX
GPIO2_D0-GPIO3_B5	GPIO.0	11	12	GPIO.1	GPIO2_D2-GPIO3_B0
GPIO2_C7-GPIO3_B4	GPIO.2	13	14	GND	GND
GPIO2_C0-GPIO3_A2	GPIO.3	15	16	GPIO.4	GPIO2_D5-GPIO3_C3
VCC 3V3	3.3V	17	18	GPIO.5	GPIO4_C6-GPIO3_D3
SPI1-SPI4-MOSI 3V3	MOSI	19	20	GND	GND
SPI1-SPI4-MISO 3V3	MISO	21	22	GPIO.6	GPIO2_D4-GPIO3_C2
SPI1-SPI4-CLK 3V3	SCLK	23	24	CE0	SPI1-SPI4-CS 3V3
GND	GND	25	26	CE1	GPIO4_C7-GPIO3_D2
I2C5-I2C7-SDA 3V3	SDA.0	27	28	SCL.0	I2C5-I2C7-SCL 3V3
GPIO2_D7-GPIO3_B6	GPIO.21	29	30	GND	GND
GPIO2_D6-GPIO3_A6	GPIO.22	31	32	GPIO.26	GPIO2_C4-GPIO3_A1
GPIO2_C5-GPIO3_A4	GPIO.23	33	34	GND	GND
GPIO2_C3-GPIO3_A0	GPIO.24	35	36	GPIO.27	GPIO2_C2-GPIO3_A5
GPIO2_C1-GPIO3_A3	GPIO.25	37	38	GPIO.28	GPIO2_D3-GPIO3_B1
GND	GND	39	40	GPIO.29	GPIO2_D1-GPIO3_A7

Header20X2_2.54SPZ

图 2.2.3.2.1 ELF2 开发板及 40pin 排针原理图

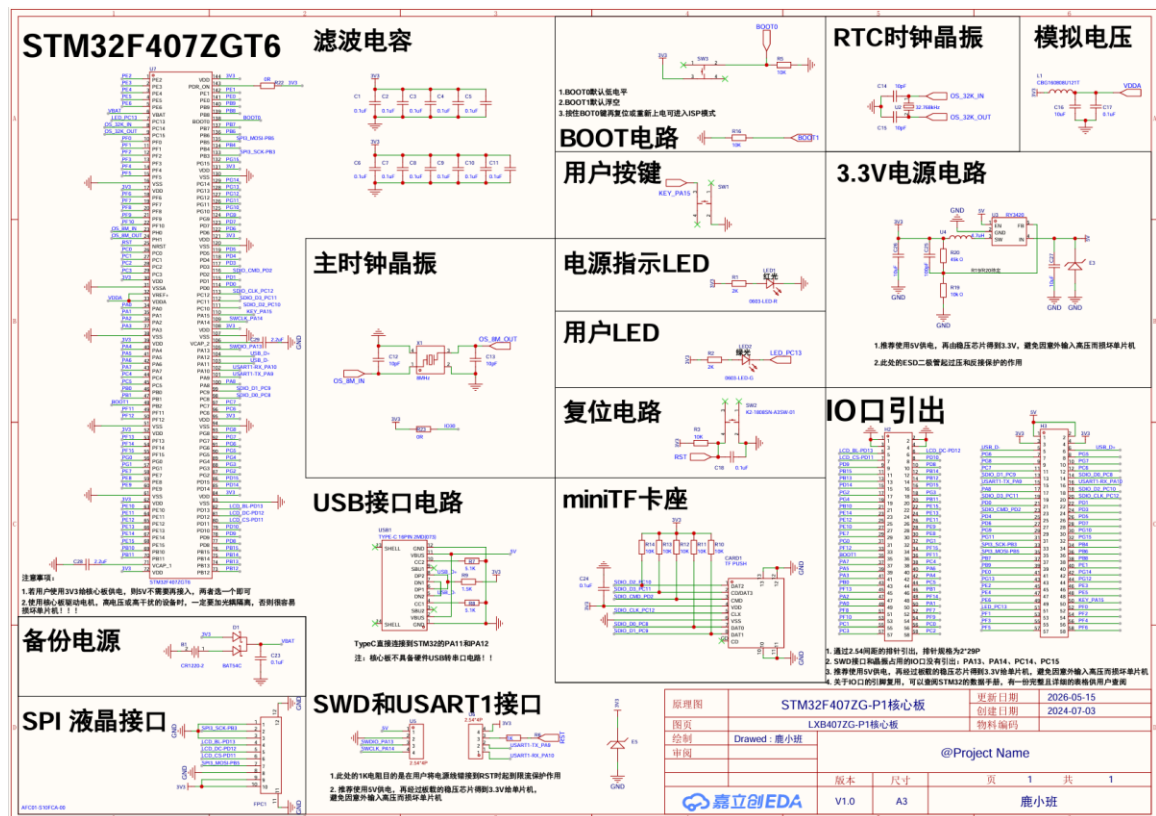


图 2.2.3.2.2 STM32F407ZGT6 及引脚总图

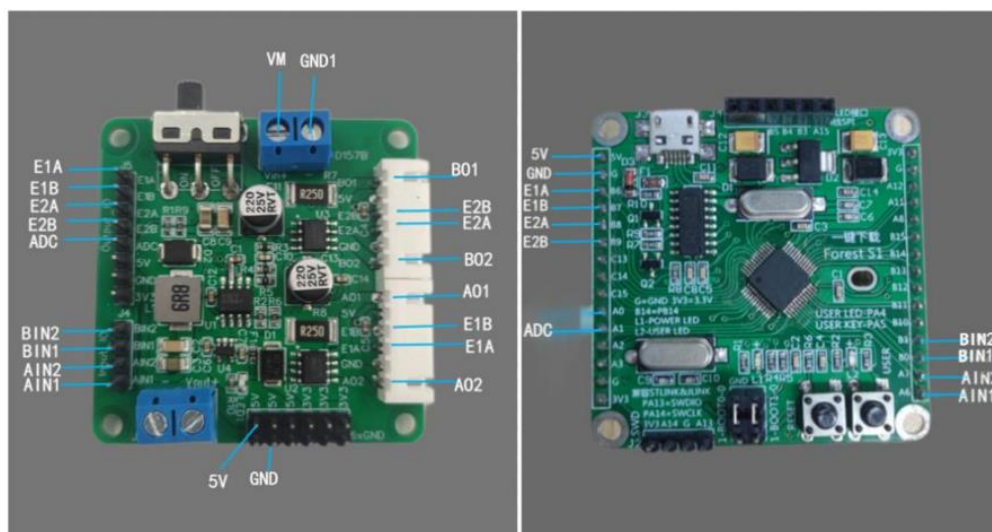


图 2.2.3.2.3 AT8236 引脚图

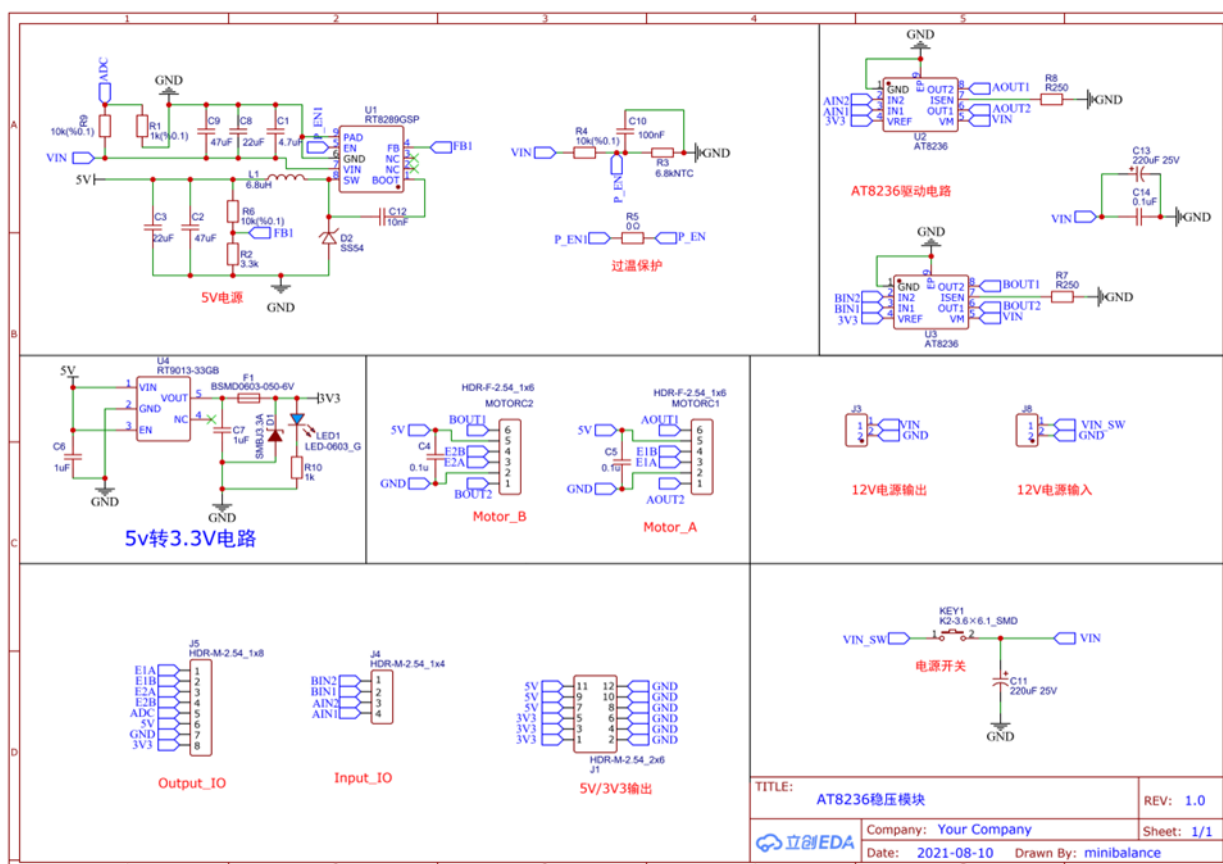
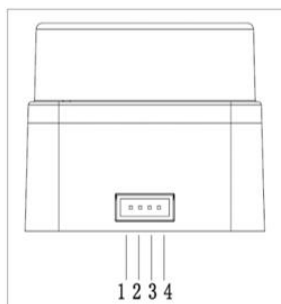


图 2.2.3.2.3 AT8236 电路原理图

LD06 uses ZH1.5T-4P 1.5mm connectors to connect with external system to implement power supply and data receiving. Below please find the definition and parameter requirements

for specific interfaces:



Number	Signal Name	Type	Description	Min	Typical	Max
1	Tx	Output	Radar data output	0V	3.3V	3.5 V
2	PWM	Input	Motor control signal	0V	-	3.3 V
3	GND	Power Supply	Power negative	-	0V	-
4	P5V	Power Supply	Power positive	4.5V	5V	5.5 V



图 2. 2. 3. 2. 4 LD06 雷达引脚定义图

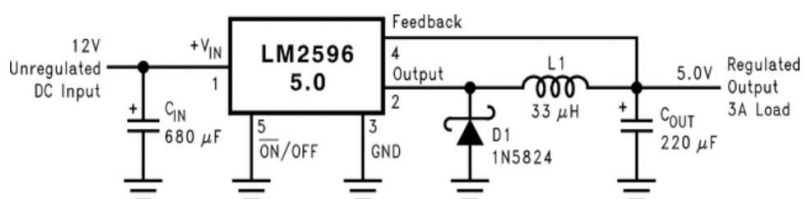


图 2. 2. 3. 2. 5 稳压模块及电路原理图

2.3 软件系统介绍

2.3.1 软件整体介绍（含 PC 端或云端，结合关键图片）；

一、软件概述

智能 AED 救援车监控平台是一款用于远程监控搭载 AED（自动体外除颤器）救援机器人运行状态的桌面/移动端软件。系统通过 WebSocket 连接 ROS2 机器人系统，提供实时地图渲染、视频监控、状态遥测与智能报警等一体化监控功能。

二、技术栈

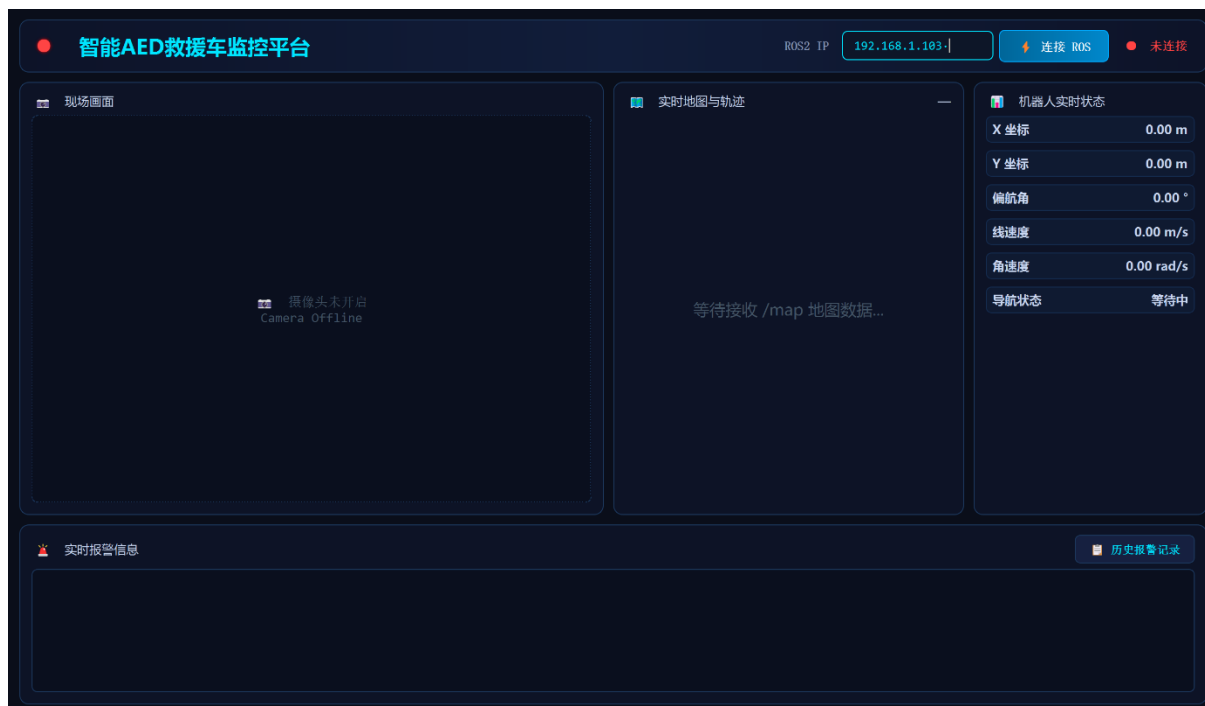
类别	技术
编程语言	Python3
桌面 GUI	PyQt5
机器人通信	roslibpy（通过 rosbridge_server 的 WebSocket 协议连接 ROS2）
计算机视觉	OpenCV+YOLOv8
AI 推理	YOLOv8 跌倒检测模型（fall.rknn）
数据持久化	SQLite（报警记录存储）
数值计算	NumPy
打包工具	PyInstaller（打包为 EXE）

三、项目结构

```

AED_UI/
├──main.py#程序入口
├──ui/
│   ├──main_window.py#主窗口（三栏布局+报警系统）
│   └──alarm_window.py#历史报警弹窗（分页查询）
├──widgets/
│   ├──map_widget.py#地图渲染（OccupancyGrid+LaserScan+Nav2Path）
│   └──camera_widget.py#RTSP 摄像头+YOLO 跌倒检测
├──ros/
│   └──ros_client.py#ROS2WebSocket 通信线程
├──database/
│   └──db.py#SQLite 数据库管理
├──kivy_build/#Android/Kivy 版本
│   ├──main.py#Kivy 主程序
│   ├──aed.kv#Kivy 布局文件
│   ├──ros_client.py#Kivy 版 ROS 客户端
│   └──buildozer.spec#Buildozer 打包配置
├──best.onnx/best.pt#YOLOv8 跌倒检测模型
├──logo.jpg/logo.ico#应用图标
└──打包为 EXE.bat#PyInstaller 打包脚本
  
```


四、界面布局



五、使用流程

- 1.启动程序→自动尝试连接 RTSP 摄像头
- 2.输入机器人 IP 地址→点击「连接 ROS」
- 3.连接成功后自动接收地图、位姿、激光、路径等数据
- 4.实时监控机器人运行状态和现场画面
- 5.发生跌倒/导航失败等紧急事件→界面自动切换红色警报模式
- 6.可随时查看历史报警记录

2.3.2 软件各模块介绍（根据总体框图，给出各模块的具体设计说明。从顶层到底层逐次给出各函数的流程图及其关键输入、输出变量）；

1.ROS2 机器人通信（ros/ros_client.py）

通过 roslibpy 以 WebSocket 方式连接机器人 rosbridge_server（默认端口 9090）

订阅 8 个 ROSTopic:

/amcl_pose—AMCL 全局定位（机器人位姿）

/odom—里程计（速度信息）

/scan—激光雷达点云

/map—栅格占据地图

/plan—Nav2 全局规划路径

/navigate_to_pose/_action/status—导航状态反馈

/fall_detection//aed_fall_detection—跌倒检测（双 Topic 兜底）

/fall_event—跌倒报警开关

支持发布导航目标到/goal_pose

2.实时地图渲染（widgets/map_widget.py）

基于 PyQtQPainter 原生绘制，实现 ROS 世界坐标→Qt 像素坐标的完整坐标变换

渲染内容：

栅格地图底图：解析 OccupancyGrid，颜色映射（未知=灰色、空闲=白色、障碍=黑色、概率值=灰度渐变）

激光雷达点云：红色散点，实时反映环境障碍物

Nav2 全局路径：蓝色虚线，显示导航规划路线

机器人图标：蓝色圆形+黄色方向箭头

图例：右上角标注

交互方式：滚轮缩放+鼠标拖拽平移，首次加载自动适配视图

3.摄像头与 AI 跌倒检测（widgets/camera_widget.py）

支持 RTSP 网络摄像头（默认地址 rtsp://admin:@192.168.1.86）或 USB 摄像头
集成 YOLOv8 跌倒检测模型，支持 ONNX 和 PT 两种格式

检测逻辑：

连续 8 帧检测到跌倒→触发红色报警

报警冷却时间 5 秒（避免重复报警）

几何规则过滤（宽高比、最小尺寸等）

画面实时标注检测框和跌倒计数

4.机器人状态面板

右侧面板实时显示 6 项关键遥测数据：

指标	来源
X/Y 坐标	AMCL 全局定位
偏航角	AMCL
线速度	里程计
角速度	里程计

导航状态	Nav2Action 反馈
------	---------------

导航状态带颜色区分：执行中=青色、成功=绿色、失败=红色。

5.多级报警系统

报警等级

- 红色报警（严重）
- 橙色警告（中等）
- 黄色提醒（轻度）

触发源

YOLO 跌倒检测→红色报警

导航失败（ABORTED）→红色报警

激光避障（已注释禁用，可重新启用）

视觉反馈

触发红色报警时，整个界面切换为红色警报主题（深红背景、红色边框、红色标题），顶部显示警报横幅

自动恢复：红色报警 10 秒无新触发后自动恢复正常界面

历史记录

SQLite 存储+分页查询弹窗（每页 15 条）

第三部分完成情况及性能参数

阐述最终实现的成果（图文结合，实物照片为主）

3.1 整体介绍（整个系统实物的正面、斜 45° 全局性照片）



图 3.2.1.2 正面照片

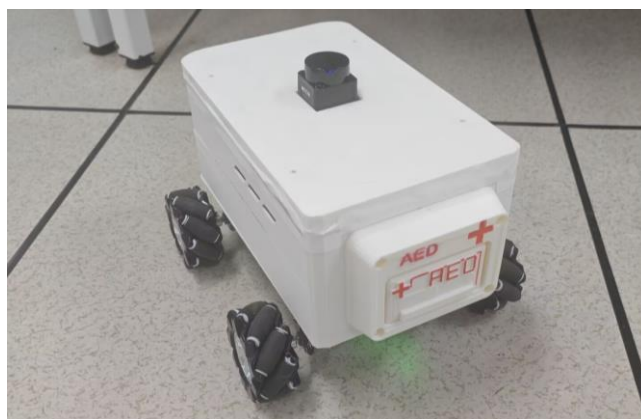


图 3.2.1.2 斜面照片



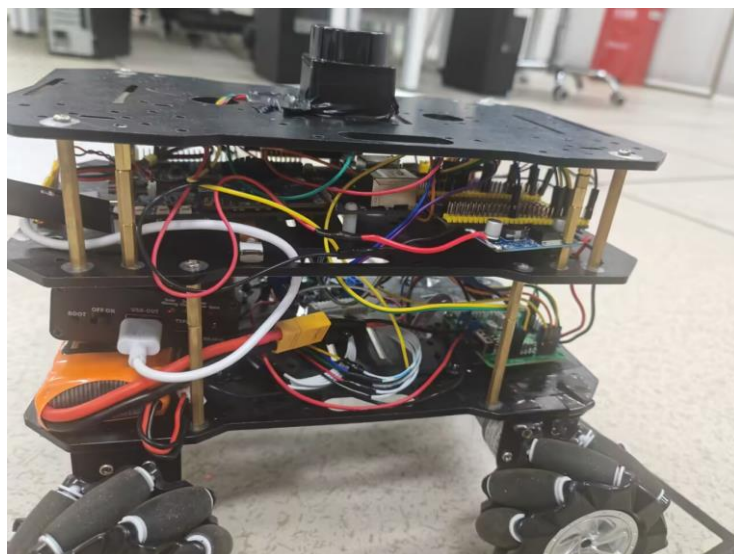
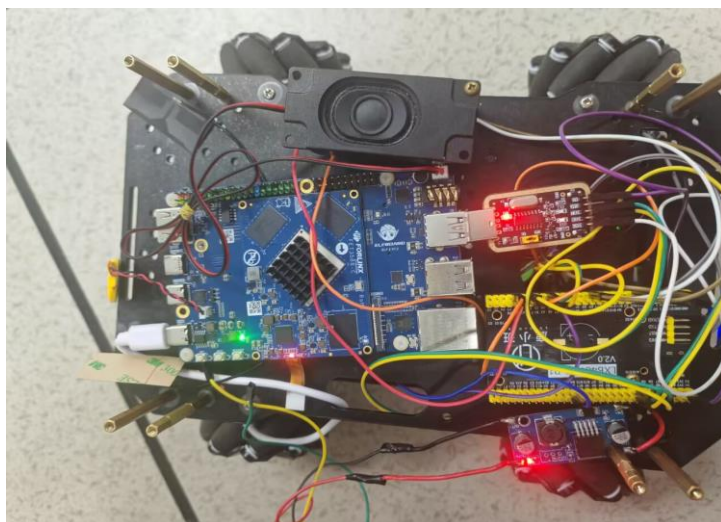
图 3.2.1.3 远程摄像头照片

3.2 工程成果（分硬件实物、软件界面等设计结果）

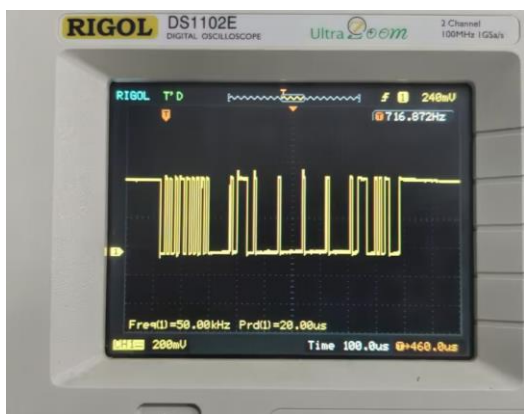
3.2.1 机械成果；（实物照片）



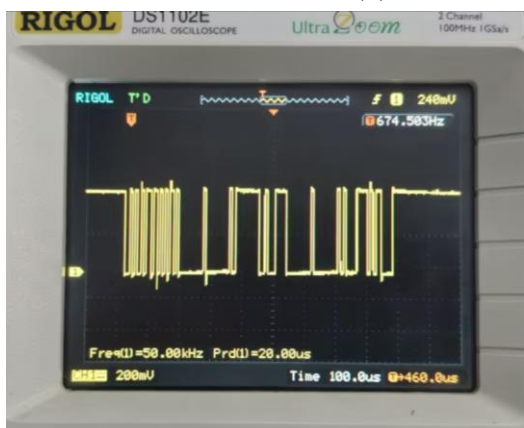
3.2.2 电路成果；（实物照片）



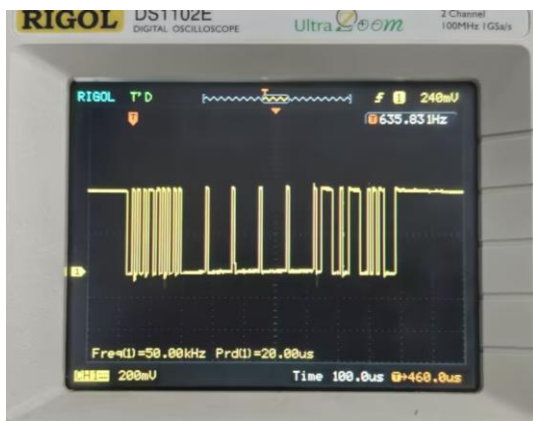
测量不同动作时引脚的波形如下：



3.3(1).1 左图为前进波形，右图为后退波形



3.3(1).2 左图为左平移波形，右图为右平移波形



3.3(1).3 左图为左转向波形，右图为右转向波形

通过观察示波器测量的小车不同运动工况下电机驱动 PWM 波形，前进、后退、平移、转向对应的脉冲时序、占空比调节、正反相位控制均符合设计预期；波形无畸变、周期稳定，验证 STM32 定时器 PWM 输出、编码测速反馈、PID 差速闭环控制算法工作可靠，底盘可精准完成全向运动动作，速度误差控制在设计允许区间，满足智能 AED 救援车自主导航运动控制需求。

(2) 激光雷达建图与自主导航

首先正确连接 LD06 雷达之后，测量雷达的输出波形，波形正常。

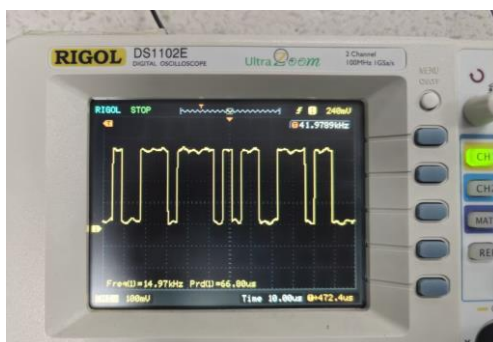


图 3.3. (2) 1 雷达输出波形

利用 LD06 激光雷达完成环境建图，并基于 ROS2Nav2 实现自主导航。系统能够自动规划最优路径，并在存在障碍物情况下完成动态避障。首先启动底盘与雷达驱动，检查 `ros2run tf2_tools view_frames`，确认 TF 树完整且无断裂。打开 SLAM 在 `ros` 的 `map` 节点上建图，通过编译的文件控制小车不断移动，慢慢在虚拟机 `rviz2` 软件上出现地图的轮廓，跑完整后将地图保存。

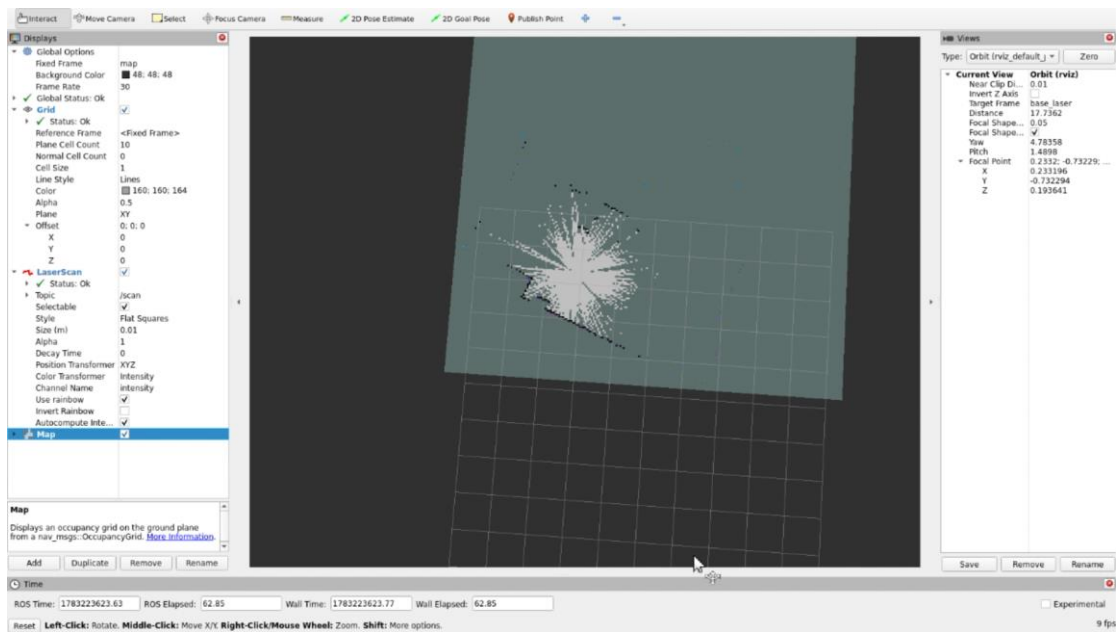


图 3.3. (2) 2rviz2 界面跑图

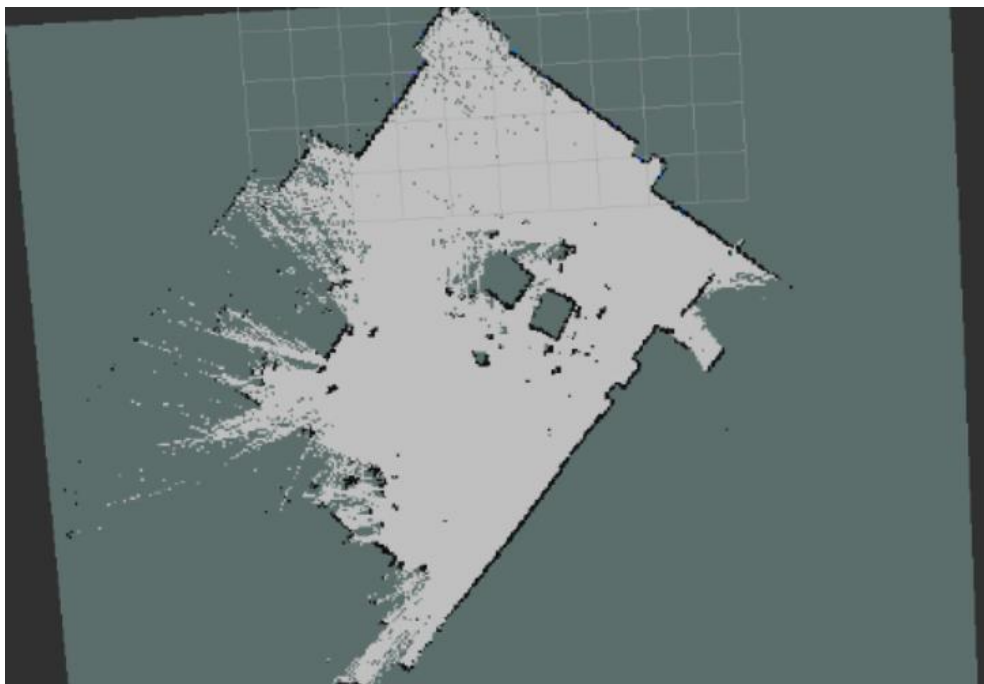


图 3.3. (2) 3 跑图过程

由于地图左下角杂物较多，小车无法进入，所以左下角有点空旷，其余的边缘轮廓明显，效果较好。



图 3.3. (2) 3 生成地图

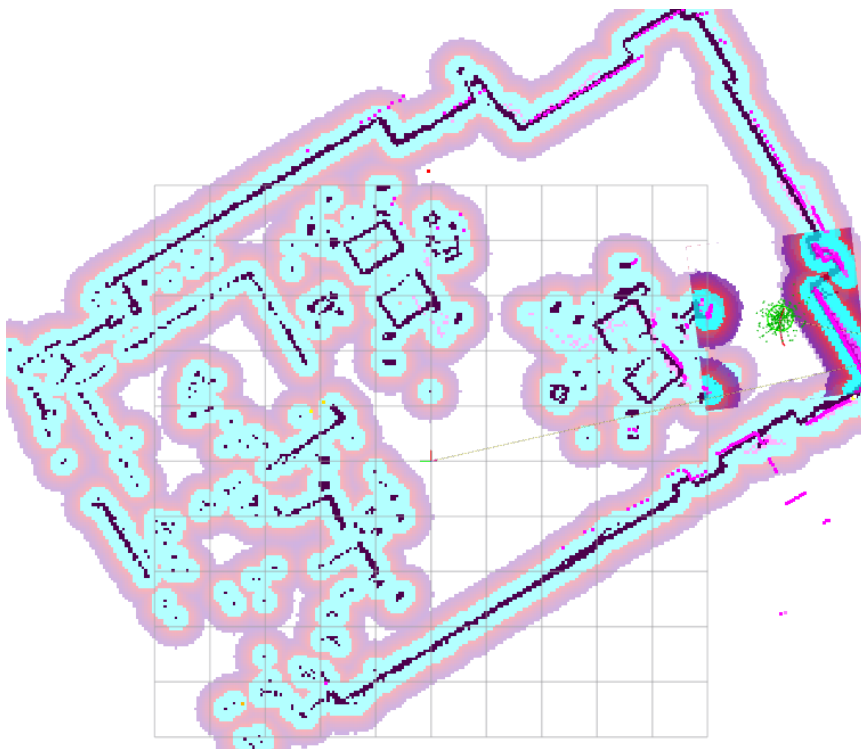


图 3.3. (2) 4 加载地图

载入地图后，可以清晰地看到小车当前位置和雷达点云，障碍的周围一圈为障碍物膨胀安全缓冲区，是导航路径规划的避障安全距离。

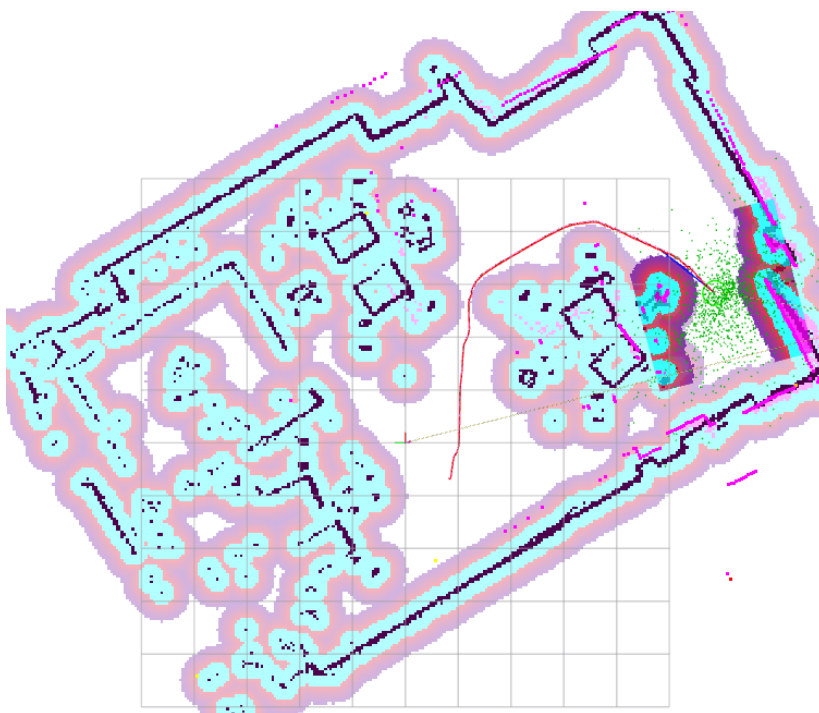


图 3.3. (2) 5 路径规划

接入 Nav2, 确定目标点后，小车在地图上会显示智能规划路径并按照路线行驶。

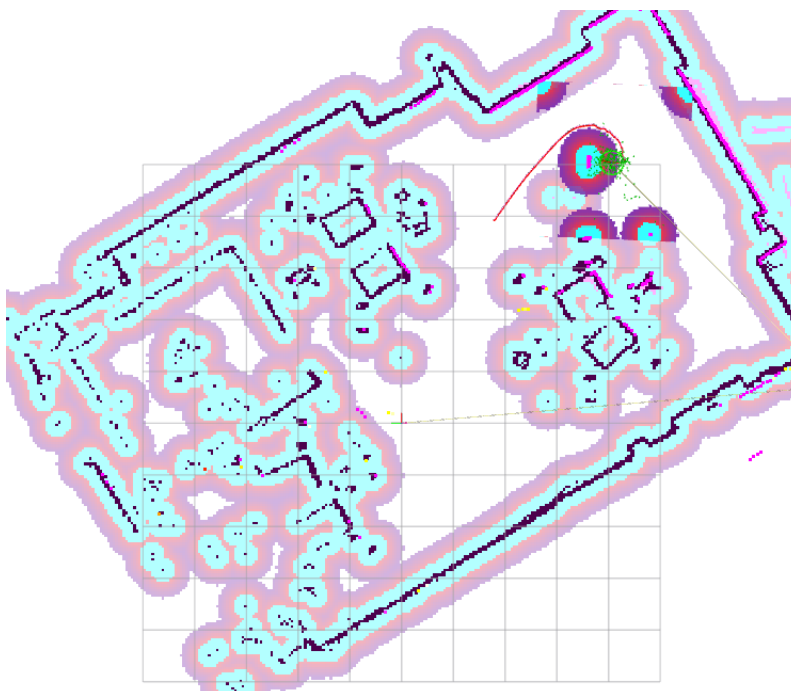


图 3.3. (2) 6 实时避障

面对地图上出现的障碍，雷达探测到之后返回 elf2 开发板，能利用算法进行实时避障。

```
elf@elf2-desktop:~$ ros2 topic echo /amcl_pose --once
header:
  stamp:
    sec: 1783411614
    nanosec: 791696990
  frame_id: map
pose:
  pose:
    position:
      x: -0.3376132159520523
      y: -1.0516838927667977
      z: 0.0
    orientation:
      x: 0.0
      y: 0.0
      z: 0.5280769539277482
      w: 0.8491965206772758
  covariance:
    - 0.05749152900917954
    - -0.017339287464425975
    - 0.0
    - 0.0
    - 0.0
    - 0.0
    - -0.017339287464425975
    - 0.12333217152185705
```

图 3.3. (2) 7 AMCL 定位

通过订阅 AMCL 输出的/amcl_pose 话题，可实时读取机器人在全局栅格地图中的二维坐标与航向角，通过编写程序加入目的地坐标和航向角，无需在 rviz2 软件中调度，让小车自动驶向指定目标点。

(3) 跌倒检测模型部署

采用 PyTorch 完成模型训练，经转换 ONNX，再次转换为 rknn 后部署至 RK3588NPU，实现边缘端实时跌倒检测。

```

Anaconda Prompt (anaconda) x + -
5      -1 1      73984 ultralytics.nn.modules.conv.Conv      [64, 128, 3, 2]
6      -1 2      197632 ultralytics.nn.modules.block.C2f      [128, 128, 2, True]
7      -1 1      295424 ultralytics.nn.modules.conv.Conv      [128, 256, 3, 2]
8      -1 1      460288 ultralytics.nn.modules.block.C2f      [256, 256, 1, True]
9      -1 1      164608 ultralytics.nn.modules.block.SPPF      [256, 256, 5]
10     -1 1      0      torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
11     [-1, 6] 1      0      ultralytics.nn.modules.conv.Concat      [1]
12     -1 1      148224 ultralytics.nn.modules.block.C2f      [384, 128, 1]
13     -1 1      0      torch.nn.modules.upsampling.Upsample      [None, 2, 'nearest']
14     [-1, 4] 1      0      ultralytics.nn.modules.conv.Concat      [1]
15     -1 1      37248  ultralytics.nn.modules.block.C2f      [192, 64, 1]
16     -1 1      36992  ultralytics.nn.modules.conv.Conv      [64, 64, 3, 2]
17     [-1, 12] 1      0      ultralytics.nn.modules.conv.Concat      [1]
18     -1 1      123648 ultralytics.nn.modules.block.C2f      [192, 128, 1]
19     -1 1      147712 ultralytics.nn.modules.conv.Conv      [128, 128, 3, 2]
20     [-1, 9] 1      0      ultralytics.nn.modules.conv.Concat      [1]
21     -1 1      493056 ultralytics.nn.modules.block.C2f      [384, 256, 1]
22     [15, 18, 21] 1 751702 ultralytics.nn.modules.head.Detect      [2, [64, 128, 256]]
Model summary: 225 layers, 3011238 parameters, 3011222 gradients, 8.2 GFLOPs

Transferred 319/355 items from pretrained weights
Freezing layer 'model.22.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks with YOLOv8n...
AMP: checks passed
train: Scanning E:\super ubuntu\fall\train\labels.cache... 4590 images, 979 backgrounds, 0 corrupt: 100%|██████████| 45
val: Scanning E:\super ubuntu\fall\valid\labels.cache... 1194 images, 185 backgrounds, 0 corrupt: 100%|██████████| 1194
Plotting labels to runs\detect\train10\labels.jpg...
optimizer: AdamW(lr=0.001, momentum=0.937) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0005), 63 bias(decay=0.0)
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to runs\detect\train10
Starting training for 150 epochs...

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/150   4.3G      1.556     2.769     1.699      38          640: 22%|██████| 31/144 [00:20<01:07, 1

```

图 3.3. (3) .1yolov8 模型训练



图 3.3. (3) .2 跌倒模型训练结果

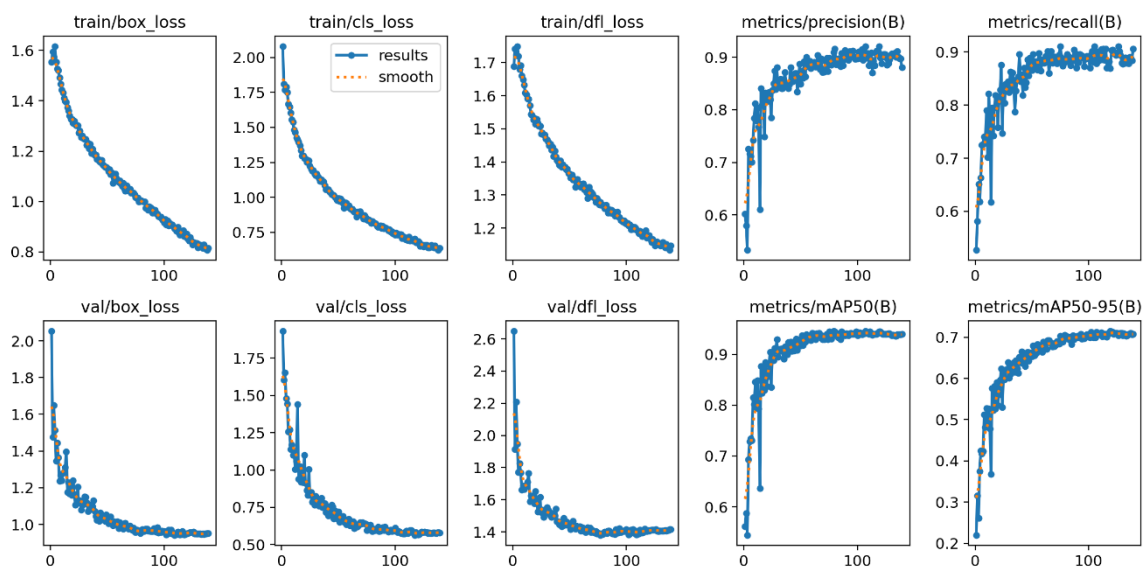


图 3.3. (3) .2 跌倒模型学习曲线与性能指标图

子图标签	含义	说明
train/box_loss	训练集边界框回归损失	衡量预测框与真实框的坐标误差（如 CIoU、GIoU 等）。下降趋势正常，说明模型在学习定位。
train/cls_loss	训练集分类损失	衡量类别预测的交叉熵误差。持续下降→分类能力提升。
train/df1_loss	训练集分布焦点损失 (DistributionFocalLoss)	常见于 YOLOv8/v9 等使用 DFL 的模型，用于优化边界框回归的分布建模（替代传统 IoUloss），使回归更平滑稳定。
metrics/precision(B)	验证集精度 (Precision@IoU=0.5)	正确检出的目标中，真正属于正类的比例（避免误检）。值越高越好，收敛到~0.9 表示高精度。
metrics/recall(B)	验证集召回率 (Recall@IoU=0.5)	所有真实目标中被成功检出的比例（避免漏检）。值高说明覆盖全面。
val/box_loss	验证集边界框损失	若持续高于训练损失但同步下降→泛化良好；若突然上升→过拟合或数据分布偏移。
val/cls_loss	验证集分类损失	同上，应平稳下降。
val/df1_loss	验证集 DFL 损失	关键指标：若验证 DFL 损失显著高于训练（如>1.5 倍），可能过拟合；此处趋势一致，健康。

metrics/mAP50(B)	验证集平均精度 (mAP@0.5)	核心综合指标：IoU 阈值为 0.5 时的平均精度。 >0.9 表示模型非常优秀（常见于 COCOval 上 YOLOv8n 可达 0.38, 但你的任务可能是小数据集/简单场景，故更高）。
metrics/mAP50-95(B)	验证集平均精度 (mAP@[0.5:0.95])	更严格的指标：IoU 从 0.5 到 0.95 步长 0.05 取平均。反映模型对定位精度要求高的鲁棒性。值 ≈ 0.7 表明模型在高 IoU 下仍较可靠。

模型训练成功，损失函数全部收敛所有 train 和 val 损失均随 epoch 下降并趋于稳定（无震荡、无发散），说明优化稳定。训练/验证损失差距小，无明显过拟合。

性能指标上：

mAP50 ≈ 0.92 ，几乎所有目标在 IoU ≥ 0.5 时都能正确检出；

mAP50-95 ≈ 0.70 ，即使要求严格（IoU ≥ 0.75 ），仍有较高检出质量；

Precision&Recall 均 >0.85 ，平衡了“少漏检”和“少误检”。

DFL 损失表现健康，dfl_loss 在训练/验证中同步下降，且验证损失未异常升高，说明 DFL 机制有效提升了回归稳定性，没有因分布建模引入噪声。

编写 python 文件验证模型。



图 3.3. (3) .3 跌倒模型 fall.pt 验证结果

经过初步的验证，训练的跌倒 fall.pt 模型可以正确识别人物的跌倒，下一步就是在 x86 平台把 pt 模型转化格式为 onnx 格式，再在虚拟机上把 onnx 格式模

型转化为 rknn 格式。

经过转化之后，rknn 格式的模型可以正常运行在 ELF2 开发板上，使用图片进行初步验证。

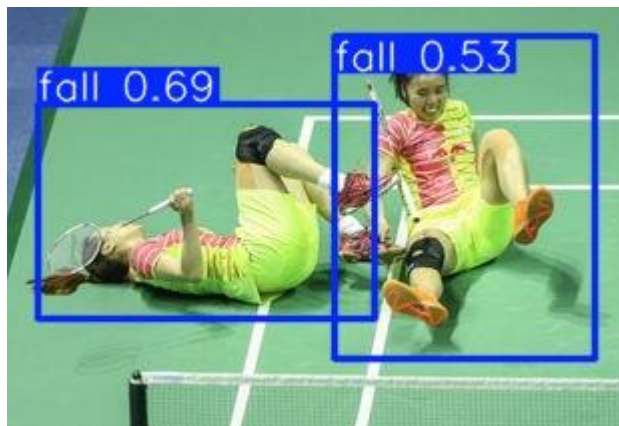


图 3.3. (3) .4 跌倒模型 fall.rknn 图片验证结果

之后，我们通过网络串流，使 ELF2 开发板可以获取远程摄像头的视频流，并编写 python 文件对摄像头回传的图像进行实时的跌倒检测。

由于小车使用 wifi 连接，经过实验分析后，发现回传主机的模型检测结果存在一定的网络延迟，为了结果绝对准确，我们在 ELF2 开发板外接了屏幕进行验证。



图 3.3. (3) .5 跌倒模型 fall.rknn 视频多次验证结果

经过反复实验验证，fall.rknn 跌倒检测模型可以完美运行在 ELF2 开发板上。

第四部分总结

4.1 可扩展之处

环境感知方面，目前主要依赖 LD06 二维激光雷达完成建图、定位及避障，后续可集成 3D 深度相机，识别障碍物高度、体积和空间结构，弥补二维激光雷达对悬空及低矮障碍物的感知不足；同时，可增加超声波测距模块，与激光雷达融合，实现近距离盲区检测，提升避障精度。

运动控制方面，可集成惯性测量单元（IMU），实时检测车体姿态与地形坡度，自适应调整驱动参数，确保在斜坡、颠簸等复杂地形下的行驶稳定性。

此外，系统还可扩展多目标识别、多车协同调度、5G 远程控制及云平台数据管理，满足智慧医院、智慧校园及大型公共场所应急救援需求，为智能急救系统提供技术基础。

4.2 心得体会

本项目从立项到完成经历了需求分析、硬件搭建、底层驱动开发、人工智能模型训练、自主导航部署以及整车联调等多个阶段，是一次理论与实践深度结合的开发过程，也让团队成员对嵌入式系统、机器人控制和人工智能技术有了更加全面的认识。

项目初期，我们首先完成了 STM32 底层控制系统的开发，实现了 PWM 驱动、编码器测速、电机 PID 闭环控制等基础功能。由于编码器测速受采样周期、机械误差以及参数整定等因素影响，车辆初期运行存在速度不一致、直线行驶偏移等问题。我们通过反复调整 PID 参数，优化测速算法，并结合大量实验不断改进控制策略，使底盘运动逐渐趋于稳定，为后续自主导航提供了可靠的运动基础。

在完成底层控制后，我们开始搭建 ROS2 开发环境，利用 LD06 激光雷达完成环境建图、定位及导航功能。由于激光雷达安装角度、底盘运动误差以及地图质量都会影响导航效果，我们多次调整雷达安装位置，优化建图流程，并不断修改导航参数，最终实现了较稳定的自主导航和自动返航功能。

人工智能部分是本项目难度最大的内容之一。我们自行采集并整理跌倒检测数据集，利用 PyTorch 完成模型训练，并不断调整网络参数，提高模型识别准确率。随后按照 PT 模型、ONNX 模型到 RKNN 模型的流程完成模型转换，并部署

到飞凌 ELF2（RK3588）开发板的 NPU 进行推理。在模型部署过程中，我们遇到了算子不兼容、模型转换失败以及误识别等问题，通过查阅官方文档、分析日志和不断调试，最终成功实现模型在边缘设备上的实时运行。

系统联调阶段也是整个项目最具挑战性的环节。由于 STM32、ELF2、激光雷达、语音播报等多个模块需要协同工作，一个模块出现异常都会影响整车运行。团队针对串口通信、ROS 节点通信、导航控制及任务调度进行了大量测试，对系统进行了多轮优化，提高了整体稳定性和响应速度。

通过本项目的研发，我们不仅掌握了 STM32 嵌入式开发、ROS2 机器人开发、激光雷达导航、深度学习模型训练与部署等关键技术，也进一步提升了查阅资料、自主学习、团队协作以及分析和解决工程问题的能力。未来，我们计划继续完善系统功能，引入 3D 深度相机、多传感器融合和智能调度算法，不断提高系统的智能化水平，为公共应急救援提供更加高效、可靠的解决方案。

第五部分参考文献

- [1]STMicroelectronics.*STM32F103x8/BSTM32F103xC/D/EReferenceManual(RM0008)*[EB/OL].STMicroelectronics,2024.
- [2]STMicroelectronics.*STM32F103x8/BDatasheet*[EB/OL].STMicroelectronics,2024.
- [3]RockchipElectronicsCo.,Ltd.*RK3588TechnicalReferenceManual*[EB/OL].Rockchip,2024.
- [4]RockchipElectronicsCo.,Ltd.*RKNNToolkit2UserGuide*[EB/OL].Rockchip,2024.
- [5]保定飞凌嵌入式技术有限公司.*ELF2 (RK3588) 开发板用户手册*[M/OL].保定：飞凌嵌入式，2024.
- [6]保定飞凌嵌入式技术有限公司.*RK3588 开发板硬件设计与开发资料*[M/OL].保定：飞凌嵌入式，2024.
- [7]EAITechnology.*LD06LiDARUserManual*[EB/OL].EAITechnology,2024.
- [8]QuigleyM,ConleyK,GerkeyB,etal.ROS:AnOpen-SourceRobotOperatingSystem[C].ICRAWorkshoponOpenSourceSoftware.Kobe,Japan,2009.
- [9]MacenskiS,SinghS,MartinF,etal.TheMarathon2:ANavigationSystem[J].*JournalofOpenSourceSoftware*,2020,5(52):2515.
- [10]PaszkeA,GrossS,MassaF,etal.PyTorch:AnImperativeStyle,High-PerformanceDeepLearningLibrary[C].*AdvancesinNeuralInformationProcessingSystems*.2019.
- [11]BaiJ,LuF,ZhangK,etal.ONNX:OpenNeuralNetworkExchange[EB/OL].LinuxFoundation,2019.
- [12]JocherG,ChaurasiaA,QiuJ.*UltralyticsYOLO*[EB/OL].Ultralytics,2024.
- [13]GoodfellowI,BengioY,CourvilleA.*DeepLearning*[M].Cambridge:MITPress,2016.
- [14]ThrunS,BurgardW,FoxD.*ProbabilisticRobotics*[M].Cambridge:MITPress,2005.
- [15]AstromKJ,MurrayRM.*FeedbackSystems:AnIntroductionforScientistsandEngineers*[M].Princeton:PrincetonUniversityPress,2008.
- [16]胡春旭.*ROS 机器人开发实践*[M].北京：机械工业出版社，2021.